Universidad de los Andes
Facultad de Ingeniería y Ciencias Aplicadas

Semestre 2026-2
Arquitectura de Computadores
Profesor: Jorge Gómez Mir

Proyecto 4

Pochoco Trader: co-diseño hardware/software sobre Espino Core

El proyecto considera una entrega en GitHub, una competencia en vivo y una defensa presencial:

- **Fecha de entrega en GitHub: 24/11 - 10:00am**.
- **Competencia y defensa en clases: 24/11 - 10:30am**.

La FPGA maestra estará disponible en la **clase del 19/11** para que cada grupo haga pruebas de conexión y de protocolo. Utilizar los mismos grupos del proyecto anterior. El punto de partida es el **Espino Core del repositorio base** (`https://github.com/nic0villegasc/pochoco_soc`) o la versión modificada por su grupo en el Proyecto 2. Pueden modificar libremente el RTL, siempre que el procesador siga siendo Espino.

1. Objetivo

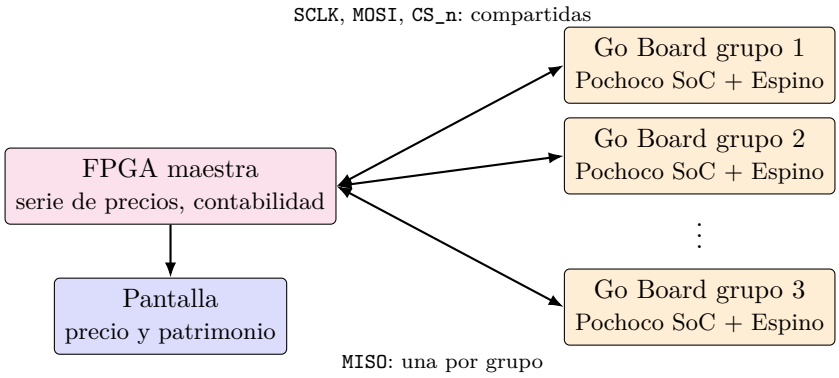
Diseñar un **agente de inversión** que corra en el Pochoco SoC sobre la Go Board. Una FPGA maestra, operada por el curso, transmitirá por SPI el precio de una acción, tick a tick. En cada tick, el programa del grupo deberá **predecir** el comportamiento del precio y responder si quiere **comprar, vender o mantener**, y en qué cantidad. La FPGA maestra graficará en tiempo real, en una pantalla, el precio de la acción y el patrimonio de cada grupo.

El foco del proyecto es el **co-diseño hardware/software**: a partir de su algoritmo, el grupo deberá decidir qué partes ejecutar en software (assembly sobre Espino) y qué partes llevar a hardware (extensiones de la ISA, periféricos o cambios en la microarquitectura), y justificar cada decisión con las métricas vistas en clases: ciclos, CPI, Ops/s, espacio utilizado en la FPGA (cantidad de LUTs) y frecuencia máxima. La ganancia en la competencia tiene un peso importante en la nota, pero también se evalúa la calidad del proceso y de la evidencia.

2. Escenario y protocolo

2.1. Conexión

La FPGA maestra comparte con todos los grupos las líneas `SCLK`, `MOSI` y `CS_n`: todos reciben el mismo precio al mismo tiempo. Cada grupo tiene su propia línea `MISO`, por lo que no es necesario dejarla en alta impedancia. Se usan los pines PMOD ya definidos en `goboard.pcf`.



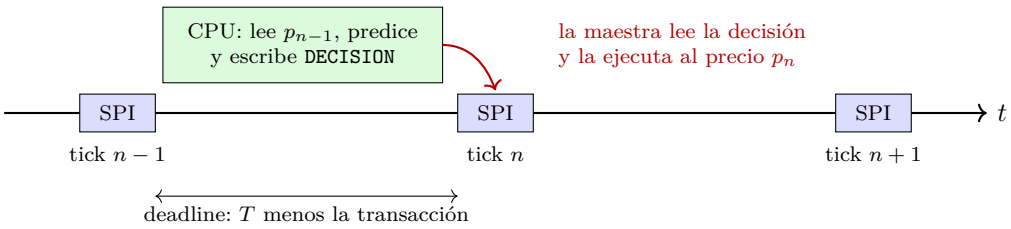
Parámetros eléctricos y de protocolo:

- SPI modo 0 (CPOL = 0, CPHA = 0), byte de 8 bits, bit más significativo primero.
- `SCLK` $\leq$ 1 MHz. El esclavo de Pochoco sincroniza `SCLK` con el reloj de 25 MHz, por lo que no debe usarse una frecuencia de SPI cercana a la del sistema.
- Ambas placas trabajan a 3,3 V y deben compartir tierra.

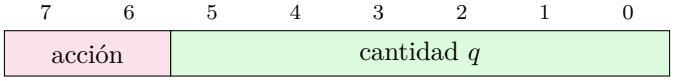
2.2. Una transacción por tick

El precio de la acción se **normaliza a un valor entero entre 1 y 255**. Cada período T, la maestra baja `CS_n` y envía el precio normalizado p_n como un byte sin signo. En la misma transacción lee por `MISO` el byte que el grupo dejó escrito en el registro `DECISION`.

Por cómo está construido `pochoco_spi_slave`, el byte que sale en la transacción n es el que estaba en `DECISION` cuando bajó `CS_n`. Es decir, es la respuesta al precio **anterior**, p_{n-1}. La maestra ejecuta esa orden al precio p_n. Así, ningún grupo puede operar con un precio que todavía no ha recibido.



2.3. Formato del byte de decisión



Campo	Bits	Significado
Acción	`[7:6]`	`00` mantener, `01` comprar, `10` vender, `11` reservado (se trata como mantener)
Cantidad	`[5:0]`	número de acciones, entre 0 y 63

La maestra ejecuta el byte que encuentre en `DECISION` al comenzar cada transacción. Si el programa no alcanzó a escribir una nueva decisión antes del siguiente tick, **se ejecuta otra vez la decisión anterior**. Cada grupo debe considerar este comportamiento en su diseño.

2.4. Reglas de mercado

- Cada grupo parte cada ronda con una caja de \$10 000 y 0 acciones.
- Una compra de q acciones cuesta $q \cdot p_n + C$, con una comisión fija $C = \$5$ por orden ejecutada. Si la caja no alcanza, se compra el máximo posible.
- Una venta entrega $q \cdot p_n - C$ y no puede superar las acciones que el grupo tiene. No se permiten ventas en corto ni deuda.
- El patrimonio en el tick n es $W_n = \text{caja}_n + \text{acciones}_n \cdot p_n$.
- La maestra lleva la contabilidad oficial. El grupo puede llevar su propia copia si su algoritmo la necesita.

Los valores de caja inicial y comisión son referenciales; los definitivos se publicarán junto con la serie de práctica.

2.5. Rondas y deadline

La competencia tiene tres rondas con series distintas. En cada una, el período entre precios es más corto, de modo que la eficiencia del programa y del hardware se transforma directamente en ganancia o pérdida:

Ronda	Período T	Ciclos disponibles a 25 MHz	Instrucciones con CPI = 2,35
1	10 ms	250 000	$\approx$ 106 000
2	1 ms	25 000	$\approx$ 10 600
3	0.1 ms	2 500	$\approx$ 1 060

A estos ciclos hay que descontar la duración de la transacción SPI y el tiempo de sincronización. Las series de evaluación son **ocultas**. Se publicará una serie de práctica, y un **testbench de ejemplo** que emula a la FPGA maestra. Deben usar el mismo programa para todas las series de tiempo.

3. Actividades

3.1. Algoritmo de predicción y decisión

- Diseñe y valide su algoritmo fuera de la placa (Python o C) con la serie de práctica. Puede usar, por ejemplo, medias móviles, momentum, regresión lineal incremental o reversión a la media, pero debe explicar la idea detrás de su elección.
- El algoritmo debe poder implementarse con los **recursos implementados en la FPGA**: aritmética entera o de punto fijo, la memoria del SoC y las LUTs disponibles, dentro del presupuesto de ciclos de cada ronda. Explique cómo decide la cantidad q.
- Compare su algoritmo con dos referencias: **no operar** y **comprar al inicio y mantener**. Reporte patrimonio final, número de órdenes, comisiones pagadas y pérdida máxima desde un peak de patrimonio.

3.2. Implementación en assembly

- Implemente su algoritmo en `trade.s` para Espino (RV32E). Describa la estructura del ciclo principal: esperar precio, actualizar estado, predecir, decidir y escribir `DECISION`.
- Documente el uso de registros y memoria, la representación de punto fijo y cómo resolvió las operaciones que Espino no tiene (los shifts están deshabilitados y no hay multiplicación ni división).
- Muestre que `trade.s` toma **exactamente las mismas decisiones** que su versión de referencia para toda la serie de práctica.
- Para cada tick, reporte instrucciones ejecutadas (IC), ciclos, CPI y **tiempo de ejecución**, en el caso promedio y en el peor caso. Compárelos con el período T de cada ronda.

3.3. Co-diseño hardware/software

- Identifique, midiendo, qué parte del algoritmo consume más ciclos. Use la ley de Amdahl para estimar cuánto podría ganar acelerándola.
- Proponga e implemente **al menos dos alternativas de hardware** y compárelas con la versión solo software. Algunos ejemplos: habilitar el shifter, un multiplicador de 8 o 16 bits, una instrucción nueva en el espacio `custom-0`, una instrucción MAC, un periférico MMIO que actualice una media móvil, mejoras al CPI del core o subir la frecuencia con el PLL.
- Para cada variante reporte: LUTs y BRAM utilizadas, frecuencia máxima, ciclos por tick, CPI medido, Ops/s (defina qué cuenta como operación útil), speedup medido contra el predicho por Amdahl y la ubicación de su kernel principal en el roofline de su Espino.
- Si sube la frecuencia con el PLL, muestre que nextpnr cierra timing a la nueva frecuencia y recalcule el presupuesto de ciclos de cada ronda.
- Elija su variante final y justifique la elección: ¿la que más gana, la más rápida o la que mejor balancea LUTs, ciclos y calidad de predicción?

3.4. Competencia en vivo

Cada grupo conectará su Go Board con la versión final a la maestra y participará en las tres rondas. Se registrará el patrimonio final de cada grupo en cada ronda. El puntaje de la competencia depende del patrimonio final respecto a la referencia *comprar y mantener* y de la posición relativa entre los grupos. El grupo de peor desempeño tendrá 0 puntos y el de mejor desempeño 2 puntos. El resto de los grupos se repartirá de forma lineal según su posición relativa.

4. Reglas

- La decisión se debe calcularla en el Espino Core en la placa. El core debe leer el precio y escribir la decisión; el hardware agregado puede acelerar partes del algoritmo, pero no reemplazar al procesador en el ciclo de decisión.
- No se puede usar información externa a la serie recibida por SPI ni almacenar series de precios en el diseño distintas de la que se está stremeando.
- Todo hardware agregado debe estar en el repositorio con su testbench.

- El README debe indicar cómo sintetizar y programar la placa, cómo ensamblar `trade.s`, cómo correr los testbenches y la versión de referencia del algoritmo, y las versiones de las herramientas usadas.

5. Defensa presencial

Durante la defensa se harán preguntas sobre cualquier parte del informe, del algoritmo, del assembly y del RTL. Todos los integrantes deben poder responder.

6. Entrega

En la entrega se considerarán los siguientes ítems:

Ítem	Puntaje
Algoritmo de predicción y comparación con las referencias (código + PDF)	1,0
Implementación en assembly y mediciones por tick: IC, ciclos, CPI y tiempo (código + PDF)	0,8
Co-diseño hardware/software: alternativas, LUTs, ciclos, CPI, Ops/s y roofline (código + PDF)	1,2
Competencia en vivo: patrimonio final en las tres rondas	2
Defensa presencial y preguntas	1
Total	**6,0**

Referencias: https://github.com/nic0villegasc/pochoco_soc; https://nandland.com/the-go-board/; https://docs.riscv.org/reference/isa/v20260120/unpriv/rv32.html; https://yosyshq.readthedocs.io/en/latest/.